DevOps Principles and Roles

Aim:

To study the fundamental principles and practices of DevOps and understand the role, responsibilities, and contributions of a DevOps Engineer throughout the software development lifecycle.

Objectives:

- To understand the basic concepts and philosophy behind DevOps.
- To study important DevOps principles such as collaboration, automation, continuous improvement, and monitoring.
- To examine the major stages involved in the DevOps lifecycle.
- To identify commonly used tools associated with different DevOps stages.
- To understand the key duties performed by a DevOps Engineer in a modern software development environment.

Hardware and Software Requirements:

Hardware

- Computer or laptop with at least 4 GB RAM; 8 GB or more is recommended.
- Intel i3 processor or equivalent and above.
- Reliable internet connection.

Software

- Windows 11 or equivalent operating system.
- Visual Studio Code or any suitable text editor.
- Web browser such as Google Chrome or Microsoft Edge.
- Git for source-code management and future DevOps experiments.

Procedure:

1. Open a web browser and study reliable resources related to DevOps principles and practices.
2. Understand the meaning of DevOps and how it integrates software development with IT operations.
3. Study major DevOps concepts including Continuous Integration (CI), Continuous Delivery (CD), and Continuous Deployment.
4. Examine the different stages of the DevOps lifecycle:

Plan → Develop → Build → Test → Release → Deploy → Operate → Monitor

5. Identify the role of a DevOps Engineer at each stage of the lifecycle.
6. Study important DevOps practices such as automation, Infrastructure as Code, automated testing, monitoring, logging, and version control.
7. Refer to suitable diagrams or infographics from reliable technical resources to obtain a better understanding of the DevOps lifecycle.
8. Record the observations and understanding obtained during the study.

Execution Steps:

1. Create a Markdown or Word document for recording the experiment.
2. Write a brief explanation of DevOps and its purpose in software development.
3. Describe the major principles followed in a DevOps environment.
4. Draw or insert a labeled representation of the DevOps lifecycle.
5. Mention the important practices used to improve software delivery and operational efficiency.
6. List at least five major responsibilities of a DevOps Engineer.
7. Associate commonly used DevOps tools with their respective lifecycle stages.
8. Save the completed document for submission and for reference in subsequent experiments.

Theory:

1. What is DevOps?

DevOps is a software development approach that brings development and operations teams together through shared practices, automation, and continuous communication.

The primary purpose of DevOps is to reduce the gap between developing software and operating it in a production environment. It encourages:

- Better coordination between development and operations teams.
- Automation of repetitive development and operational activities.
- Faster and more reliable software delivery.
- Continuous delivery of quality software.
- Reduction of errors and failures during releases.

Thus, DevOps can be viewed not merely as a collection of tools, but as a combination of culture, practices, and processes designed to improve the complete software delivery lifecycle.

2. Fundamental DevOps Principles

The major principles of DevOps include:

1. Collaboration and Communication

Development and operations teams work together rather than functioning as isolated groups. This reduces communication gaps and improves coordination.

2. Automation

Repetitive activities such as building, testing, and deployment are automated to save time, improve consistency, and minimize manual errors.

3. Continuous Improvement

Processes are regularly evaluated and refined to improve efficiency, reliability, and software quality.

4. Customer-Focused Delivery

Development and operational activities are aligned with customer requirements so that useful software can be delivered more effectively.

5. Shared Responsibility

Teams share responsibility for the software throughout its journey from development to production instead of limiting responsibility to a single department.

6. Monitoring and Measurement

Application and infrastructure performance are continuously observed. Feedback and performance metrics are used to identify problems and improve the system.

3. DevOps Lifecycle

The DevOps lifecycle consists of a sequence of interconnected stages:

Stage	Description	Example Tools
Plan	Define requirements, features, tasks, and schedules.	Jira, Trello
Develop	Write, manage, and review source code.	Git, VS Code
Build	Compile and package the application.	Maven, Gradle
Test	Perform automated and unit testing to verify quality.	Selenium, JUnit
Release	Prepare and approve software for deployment.	Jenkins, GitHub Actions
Deploy	Move applications to staging or production environments.	Docker, Kubernetes
Operate	Maintain applications and ensure their proper functioning.	Ansible, Puppet
Monitor	Observe application performance, logs, and system errors.	Prometheus, Grafana

These stages form a continuous cycle in which feedback obtained from operation and monitoring can be used to improve future planning and development.

4. Important DevOps Practices

Continuous Integration (CI)

Developers frequently integrate their code into a shared repository. This helps detect integration problems early.

Continuous Delivery (CD)

The software release process is automated so that tested code can be prepared for delivery quickly and reliably.

Infrastructure as Code (IaC)

Infrastructure configuration is managed through code instead of relying entirely on manual configuration. Tools such as Terraform and Ansible can be used for this purpose.

Automated Testing

Testing activities are incorporated into automated pipelines to identify defects efficiently and maintain software quality.

Monitoring and Logging
System and application metrics, logs, and errors are continuously collected to help identify and resolve operational problems.

Version Control
Tools such as Git maintain a history of source-code changes and support collaborative development.

5. Role of a DevOps Engineer

A DevOps Engineer acts as a bridge between software development and IT operations. The role focuses on improving automation, deployment reliability, infrastructure management, and system performance.

Major responsibilities include:

1. Designing and maintaining CI/CD pipelines for reliable software delivery.
2. Automating repetitive activities such as building, testing, and deployment.
3. Monitoring applications and infrastructure to identify performance or operational problems.
4. Managing infrastructure using Infrastructure as Code and configuration-management tools.
5. Supporting reliable and repeatable software deployments.
6. Applying appropriate security and compliance practices within development and deployment workflows.
7. Collaborating with development and operations teams throughout the software lifecycle.

Sample Command

The following command can be used to verify whether Git is installed on the system:

```
C:\Users\Amol Pathak>git --version
git version 2.53.0.windows.3
```

If Git is installed correctly, the command displays the installed Git version.

Result

The fundamental concepts of DevOps, its major principles and practices, lifecycle stages, and the responsibilities of a DevOps Engineer were studied successfully. The relationship between development, operations, automation, continuous delivery, monitoring, and shared responsibility was also understood.

Conclusion

DevOps provides a collaborative and automated approach to software development and operations. By integrating development and operational activities, it supports faster delivery, improved reliability, continuous feedback, and better software quality. A DevOps Engineer plays an important role in implementing automation, managing infrastructure, maintaining CI/CD workflows, and ensuring reliable application performance across the software lifecycle.

Roll No: 160123733031 Exp. No: 02 Date: 24/07/2026

Version Control for SCM

Aim:

To study Version Control System (VCS) tools used for Source Code Management (SCM), with particular emphasis on Git and its role in collaborative software development.

Objectives:

- To understand the purpose and working of Version Control Systems.
- To distinguish between local, centralized, and distributed version control systems.
- To study commonly used VCS tools for source code management.
- To understand the basic features and advantages of Git.
- To examine why Git is widely adopted in modern DevOps workflows.
- To learn recommended practices for maintaining an organized version-controlled repository.

Hardware and Software Requirements:

Hardware

- Computer or laptop with an internet connection..

Software

- Git for Windows or an equivalent Git installation.
- Visual Studio Code or any suitable code editor.
- GitHub account for remote repository management.
- Web browser.

Procedure:

1. Install Git and a suitable source-code editor such as Visual Studio Code.
2. Study the concept and purpose of Version Control Systems.
3. Understand the difference between local, centralized, and distributed version control models.
4. Examine popular VCS tools such as Git, Apache Subversion (SVN), Mercurial, and Perforce.
5. Study the basic working of Git repositories and how source-code changes are tracked.
6. Understand the purpose of operations such as commit, push, pull, and clone.
7. Compare Git with other commonly used VCS tools based on their storage model, speed, offline capabilities, popularity, and branching/merging support.
8. Study recommended practices for maintaining a clean and reliable repository.

Execution Steps:

1. Open Git Bash and Visual Studio Code.
2. Create a folder for a sample project.
3. Initialize the folder as a Git repository.
4. Create a sample file such as hello.txt.
5. Add the file to the Git staging area.
6. Commit the changes with a meaningful commit message.

7. Create or access a remote repository on GitHub.
8. Connect the local repository to the remote repository.
9. Push the committed changes to the remote repository.
10. Pull changes from the remote repository to verify synchronization.
11. Observe how Git maintains the history of changes made to the project.

Theory:

1. Version Control System

A Version Control System is a software tool that records modifications made to source code over time. It allows developers to maintain different versions of a project and collaborate while working on the same codebase.

A VCS provides the following benefits:

- Maintains a history of source-code modifications.
- Allows previous versions of files to be restored when required.
- Supports simultaneous contributions from multiple developers.
- Helps track who made particular changes.
- Provides a structured mechanism for managing project files.

2. Types of Version Control Systems

1. Local Version Control System (LVCS)

Versions of files are maintained within a local database on the user's computer. An example is RCS.

2. Centralized Version Control System (CVCS)

A central server stores the repository and its versions. Developers access the files from this central location. Examples include SVN and CVS.

3. Distributed Version Control System (DVCS)

Each developer maintains a complete copy of the repository, including its history. This allows many operations to be performed locally. Git and Mercurial are examples of distributed VCS tools.

3. Popular VCS Tools

Tool	Type	General Use
Git	Distributed VCS	Source-code management and DevOps workflows
Apache Subversion (SVN)	Centralized VCS	Enterprise and centralized project management
Mercurial	Distributed VCS	Source-code version management
Perforce	Centralized/enterprise-oriented	Large projects and binary-file management

4. Comparison of VCS Tools

Feature	Git	SVN	Mercurial
Storage Model	Distributed	Centralized	Distributed
Speed	Very fast	Relatively slower	Fast
Offline Work	Supported	Limited/Not supported in the same manner	Supported
Popularity	Very high	Moderate	Lower
Branching and Merging	Lightweight and flexible	More complex	Relatively simple

5. Git in DevOps

Git is widely used in DevOps because its distributed architecture supports efficient collaboration between developers. Its lightweight branching and merging capabilities make it suitable for feature development and parallel work.

Git is particularly useful because:

- Developers can work with repositories locally.
- Branches can be created and merged efficiently.
- Changes can be tracked through commits.
- It integrates well with CI/CD workflows.
- It supports collaboration through platforms such as GitHub, GitLab, and Bitbucket.
- Developers can continue working on local repositories without continuously requiring network access.

6. Basic Git Operations

The following operations form the foundation of Git-based source-code management:

Clone

Creates a local copy of an existing remote repository.

Add

Places modified or newly created files into the staging area for the next commit.

Commit

Records staged changes in the local repository along with a descriptive message.

Push

Transfers committed local changes to a remote repository.

Pull

Retrieves changes from a remote repository and integrates them into the local working repository.

7. Recommended Version Control Practices

The following practices help maintain an effective source-code repository:

1. Make commits regularly instead of accumulating a large number of unrelated changes.
2. Use clear and meaningful commit messages.
3. Create separate branches for new features and bug fixes.
4. Synchronize with the latest repository changes before beginning new work.

5. Avoid storing passwords, credentials, or other sensitive information in the repository.
6. Use .gitignore to prevent unnecessary or sensitive files from being committed.
7. Remove obsolete branches and keep the repository properly organized.

Program Code

The following Git commands demonstrate a basic local repository workflow:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin <repository-url>
git push -u origin main
```

To retrieve the latest changes from the remote repository:

```
git pull
```

To create a local copy of an existing remote repository:

```
git clone <repository-url>
```

Result

The concept and purpose of Version Control Systems for Source Code Management were studied successfully. Different VCS models and commonly used tools were examined, and the role of Git in collaborative software development and DevOps workflows was understood.

Conclusion

Version Control Systems provide an organized mechanism for tracking source-code changes, maintaining project history, and supporting collaboration among developers. Among the available tools, Git's distributed architecture, flexible branching, offline capabilities, and integration with modern DevOps platforms make it highly suitable for contemporary software development and CI/CD workflows.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

Git Installation and Operations

Aim:

To install and configure Git, create a GitHub account and repository, and perform essential Git operations for managing and synchronizing source code.

Objectives:

- To install Git and verify its successful configuration.
- To understand the relationship between a local Git repository and a remote GitHub repository.
- To create and configure a GitHub repository for source-code management.
- To perform fundamental Git operations such as status, add, commit, push, and clone.
- To establish a connection between a local repository and its remote origin.
- To verify that source files can be successfully transferred between local and remote repositories.

Hardware and Software Requirements:

Hardware

- Computer or laptop.
- Stable internet connection.

Software

- Git for Windows.
- Git Bash.
- Visual Studio Code or any suitable source-code editor.
- Web browser.
- GitHub account.

Procedure:

1. Download and install Git on the computer.
2. Open Git Bash and verify the installation.
3. Configure the Git username and email address.
4. Create a GitHub account if one is not already available.
5. Create a new repository on GitHub.
6. Create a project directory on the local system and initialize it as a Git repository.
7. Add or create the required source files in the project directory.
8. Use git status to inspect the current state of the repository.
9. Stage the required files using git add.
10. Record the staged changes using git commit.
11. Associate the local repository with the GitHub repository using git remote add origin.
12. Verify the remote connection using git remote -v.
13. Upload the committed changes to GitHub using git push.
14. Clone a remote GitHub repository into the local system using git clone.
15. Verify that the cloned repository contains the required project files.

Execution Steps:

Step 1: Install and Verify Git

Install Git for Windows using the official installer and open **Git Bash** after installation.
Verify that Git has been installed correctly using:

```
C:\Users\Amol Pathak>git --version
git version 2.53.0.windows.3
```

A successful installation displays the installed Git version.

Step 2: Configure Git

Configure the username and email that will be associated with Git commits:

```
>git config --global user.name "Your Name"
>git config --global user.email "your-email@example.com"
```

These details are used to identify the author of subsequent commits.

Step 3: Create a GitHub Repository

1. Sign in to GitHub using a web browser.
2. Create a new repository.
3. Provide a suitable repository name.
4. Create the repository.
5. Note the repository URL, which will be used to connect the local repository.

Step 4: Initialize the Local Repository

Open Git Bash and navigate to the project directory.
Initialize Git using:

```
>git init
```

This creates a Git repository within the project directory and allows Git to begin tracking changes.

Step 5: Check Repository Status

After creating or adding a source file, check the current repository state:

```
>git status
```

The command identifies files that are currently untracked, modified, or staged.
In the demonstrated execution, the Python source file appears as an **untracked file**, indicating that it has not yet been added to the staging area.
Figure 1: Checking repository status and identifying an untracked file

Step 6: Stage the Changes

Add the project files to the staging area:

```
>git add .
```

The . specifies that the changes in the current directory should be staged.
Staging allows Git to identify which changes are intended to be included in the next commit.

Step 7: Commit the Changes

Create a commit with a meaningful message:

```
>git commit -m "Added Matrix Multiplication using numpy"
```

The commit permanently records the staged changes in the local repository history.

In the demonstrated execution, the commit records the addition of the Python source file containing the matrix multiplication implementation.

Step 8: Connect the Local Repository to GitHub

Associate the local repository with the GitHub repository:

```
>git remote add origin https://github.com/your-username/your-repository.git
```

Here, origin is the conventional name assigned to the remote repository.

To verify that the remote repository has been configured correctly:

```
>git remote -v
```

The command displays the remote URL used for fetching and pushing changes.

Step 9: Push the Local Repository to GitHub

Upload the local commits to the GitHub repository:

```
>git push -u origin main
```

Here:

- origin refers to the configured remote repository.
- main specifies the branch to which the changes are being pushed.
- -u establishes the upstream relationship between the local main branch and origin/main.

The successful output confirms that the objects were transferred and that the local branch is now associated with the remote branch.

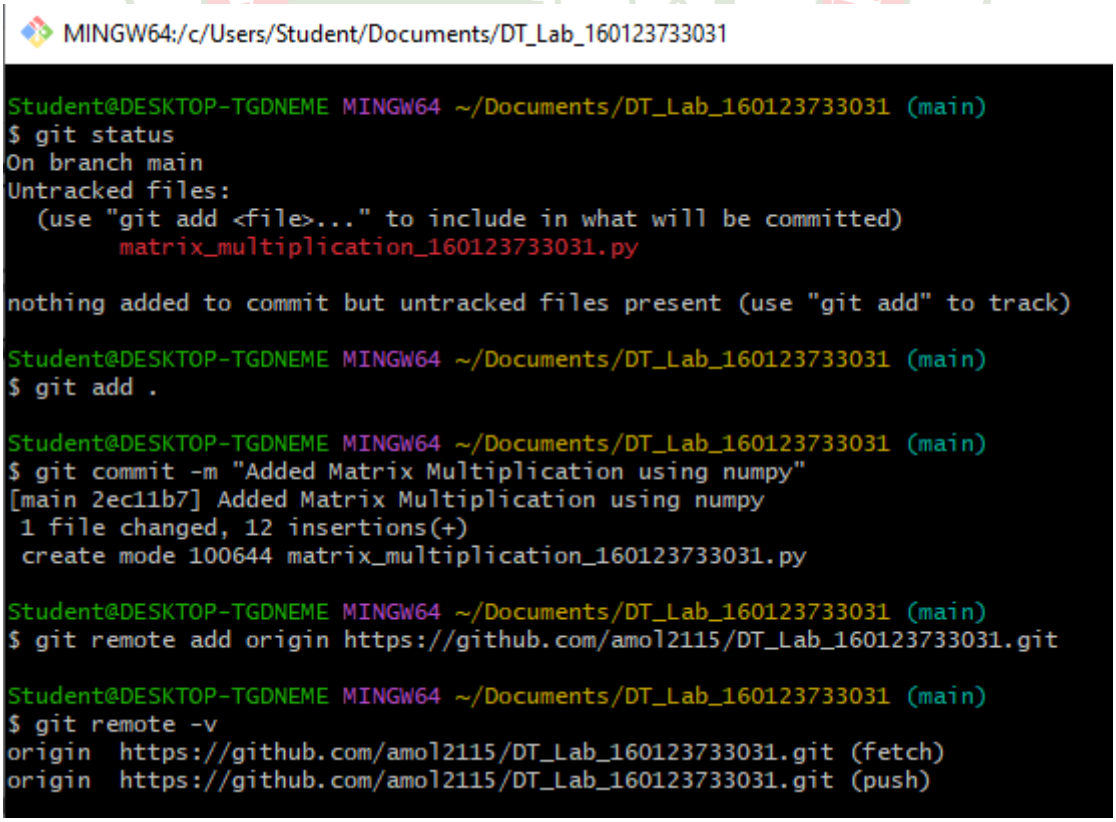


Figure 1: Verifying Status, Staging and committing project changes, adding and verifying remote repository

```
Student@DESKTOP-TGDNEME MINGW64 ~/Documents/DT_Lab_160123733031 (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 700 bytes | 350.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/amol2115/DT_Lab_160123733031.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 2: Pushing the local repository to GitHub

Step 10: Clone a Remote Repository

A repository hosted on GitHub can be copied to the local system using:

git clone https://github.com/username/repository.git

The git clone command creates a local copy of the remote repository along with its tracked files and repository history.

In the demonstrated execution, the remote repository is successfully cloned and the cloning process reports completion.

```
Student@DESKTOP-TGDNEME MINGW64 ~/Documents/DT_Lab_160123733031 (main)
$ git clone https://github.com/LingamJohnEthan29/DevOpe26-27.git
Cloning into 'DevOpe26-27'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 11 (delta 1), reused 5 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (1/1), done.
```

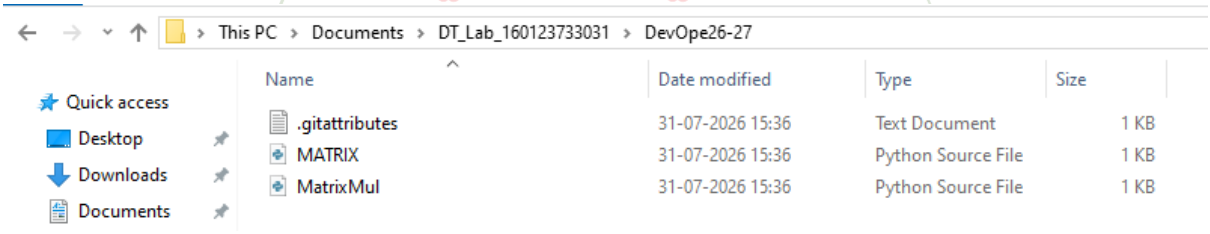
Figure 3: Cloning a GitHub repository

Step 11: Verify the Cloned Repository

Navigate into the newly created repository directory and inspect its contents:

cd repository-name

The cloned directory should contain the files stored in the remote repository.



The screenshot demonstrates the resulting local directory containing the repository files after the cloning operation.

Important Git Commands

Command	Purpose
git --version	Checks the installed Git version
git init	Creates a new local Git repository
git status	Displays the current state of tracked and untracked files
git add .	Stages changes in the current directory
git commit -m "message"	Records staged changes in the repository
git remote add origin <URL>	Connects the local repository to a remote repository
git remote -v	Displays configured remote repository URLs
git push -u origin main	Uploads local commits to the remote main branch
git clone <URL>	Creates a local copy of a remote repository

Result:

Git was successfully installed and configured, and the basic Git workflow was performed. A local repository was created, files were checked and staged, changes were committed, and the local repository was linked to a GitHub repository. The committed changes were successfully pushed to GitHub, and a remote repository was cloned to the local system.

Conclusion:

The experiment provided practical understanding of Git-based source-code management and its integration with GitHub. The operations status, add, commit, remote add, push, and clone were successfully studied and executed. These commands form the foundation for maintaining source-code versions and collaborating through remote repositories in modern DevOps workflows.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

Jenkins CI Build Job

Aim:

To install and configure Jenkins and create a simple build job in order to understand the fundamental concepts and workflow of Continuous Integration (CI).

Objectives:

- To understand the basic concept of Continuous Integration using Jenkins.
- To install and configure Jenkins on a standalone system.
- To understand the Jenkins dashboard and job management interface.
- To create and configure a basic Jenkins build job.
- To execute a build and examine its console output.
- To understand how Jenkins automates software build activities.

Hardware and Software Requirements:

Hardware

- Computer or laptop.
- Minimum 4 GB RAM; 8 GB RAM or more is recommended.
- Stable internet connection.

Software

- Windows, Linux, or macOS operating system.
- Java Development Kit (JDK) version 17 or above.
- Jenkins LTS version.
- Git, if source-code integration is required.
- Web browser such as Chrome, Firefox, or Edge.

Procedure:

1. Install JDK 17 or a later supported version and verify its installation.
2. Download and install Jenkins LTS.
3. Start the Jenkins service.
4. Open Jenkins through a web browser using the local Jenkins address.
5. Unlock Jenkins using the initial administrator password.
6. Install the suggested plugins and complete the initial configuration.
7. Create an administrator account.
8. Create a new Jenkins Freestyle project.
9. Configure a simple build step.
10. Save the job configuration and execute the job.
11. Examine the build history and console output.
12. Verify that the build completes successfully.

Execution Steps:

Step 1: Install Java

Install JDK 17 or a later version on the system.

Verify the installation using:

```
java --version
```

```
C:\Users\Amol Pathak>java --version
java 21.0.12.1 2026-08-18 LTS
Java(TM) SE Runtime Environment (build 21.0.12.1+1-LTS-4)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12.1+1-LTS-4, mixed mode, sharing)
```

Figure 1: Verification of Java installation

Step 2: Install Jenkins
Download and install the Jenkins LTS version appropriate for the operating system.
After installation, start the Jenkins service.

Step 3: Access Jenkins Dashboard
Open a web browser and navigate to:
http://localhost:8080
Jenkins uses port **8080** in the setup demonstrated in the reference material.
During the first launch, Jenkins requests the initial administrator password.

Step 4: Complete Initial Jenkins Configuration
Enter the initial administrator password and proceed with the setup.
Install the suggested plugins and create an administrator account when prompted.

Step 5: Open the Jenkins Dashboard
After completing the initial configuration, access the Jenkins dashboard.
The dashboard provides options for creating and managing Jenkins jobs.

Step 6: Create a Freestyle Project
Select: New Item → Freestyle Project → OK
Provide a suitable name for the project.
The reference experiment uses a Freestyle Project to demonstrate a basic Jenkins build workflow.

Step 7: Configure the Build Step
Within the project configuration, add a build step.
For a Windows-based system, select:
Build Steps → Execute Windows Batch Command
Enter the following command:
echo Hello Jenkins - My First Build Job
This simple command provides an easily verifiable output for the build.
The reference PDF demonstrates the same basic echo command as the build step.

Step 8: Save and Execute the Job
Save the configuration and select: **Build Now**
Jenkins creates a build for the configured project.
The build history can be viewed on the project page.

Step 9: View Console Output

Open the generated build and select:

Console Output

The console displays the commands executed by Jenkins and their corresponding output.

For the sample build, the output should contain:

```
Hello Jenkins - My First Build Job
A successful build should be reported as:
Finished: SUCCESS
```

Jenkins Build Workflow

The basic workflow demonstrated in this experiment can be summarized as:

Create Job → Configure Build Step → Save → Build Now → Console Output → Verify Success

Jenkins automates the execution of the configured build tasks, providing a basic demonstration of Continuous Integration.

Sample Build Command

echo Hello Jenkins - My First Build Job

Result:

Jenkins was successfully installed and configured, and a Freestyle build job was created and executed. The build output was examined through the Jenkins console, and successful execution of the configured build task was verified.

Conclusion:

The experiment demonstrated the fundamental workflow of Continuous Integration using Jenkins. A Jenkins environment was configured, a basic Freestyle project was created, and an automated build task was executed. The console output provided verification of the build process and demonstrated how Jenkins can automate repetitive development tasks.

Jenkins Master-Slave Setup

Aim:

To understand the Jenkins Master-Slave architecture and scale a standalone Jenkins implementation by configuring and managing slave nodes for distributed build execution.

Objectives:

- To understand the concept of distributed builds in Jenkins.
- To study the architecture and components of Jenkins Master-Slave configuration.
- To configure a Jenkins controller (master) and connect a slave (agent) node.
- To understand how Jenkins distributes build workloads across multiple nodes.
- To execute a build on a slave node and verify successful communication between the controller and agent.
- To examine the benefits of scalability, load balancing, and distributed execution in Jenkins.

Hardware and Software Requirements:

Hardware

- One system to act as the Jenkins Controller (Master).
- One or more systems to act as Jenkins Agents (Slaves).
- Network connectivity between all systems.

Software

- Windows, Linux, or macOS operating system.
- Java Development Kit (JDK 17 or above) installed on all nodes.
- Jenkins installed on the Controller node.
- Web browser.
- SSH service (recommended for Linux-based agents) or Java Web Start/Agent service.

Procedure:

1. Install Java on both the controller and agent machines.
2. Install and configure Jenkins on the controller node.
3. Verify network communication between the controller and agent systems.
4. Open the Jenkins dashboard and navigate to node management.
5. Create a new Jenkins agent node.
6. Configure the agent name, working directory, and launch method.
7. Connect the agent node to the controller.
8. Verify that the agent appears online in Jenkins.
9. Create a build job and restrict execution to the configured agent node.
10. Execute the build and verify successful execution on the agent.
11. Examine build logs and node status.
12. Record observations regarding distributed build execution.

Theory:

1. Jenkins Master-Slave Architecture

Jenkins uses a distributed architecture to improve scalability and workload management. Instead of executing all jobs on a single machine, Jenkins can distribute tasks across multiple systems.

The architecture consists of:

Controller (Master)

The controller is the central Jenkins server responsible for:

- Managing Jenkins configuration.
- Scheduling and assigning build jobs.
- Monitoring connected agent nodes.
- Maintaining build history and reports.
- Managing plugins, credentials, and users.

Agent (Slave)

An agent is a remote machine connected to the controller for executing assigned jobs.

Agent nodes:

- Receive build tasks from the controller.
- Execute builds and testing activities.
- Return execution results to the controller.
- Reduce workload on the controller machine.

2. Working of Master-Slave Architecture

The workflow can be summarized as:

Developer → Jenkins Controller → Jenkins Agent → Build Execution → Result Returned to Controller

The controller determines the availability of agent nodes and assigns jobs accordingly. The agent performs the actual execution and sends status information back to the controller.

3. Advantages of Distributed Builds

Load Distribution

Build workloads can be distributed among multiple systems instead of overloading a single server.

Scalability

Additional agents can be added whenever build requirements increase.

Platform Diversity

Different agents can be configured with different operating systems and software environments.

Faster Build Execution

Multiple jobs can execute simultaneously on separate agents.

Resource Optimization

High-resource builds can be assigned to dedicated machines.

4. Components of Jenkins Distributed Architecture

Component	Function
Jenkins Controller	Central management server
Jenkins Agent	Executes assigned jobs
Communication Channel	Connects controller and agent
Workspace	Directory where builds are executed
Executor	Build execution slot on a node

5. Agent Communication Methods

Jenkins supports multiple methods for connecting agents:

- SSH Launch Method
- Java Web Start (JNLP)
- Inbound Agent Connection
- Windows Service-based Agent
- Docker-based Agent

Among these methods, SSH and inbound agent connections are the most commonly used in production environments.

Execution Steps:

Execution Steps

Step 1: Verify Java Installation on Agent

Ensure Java is installed on the agent machine.

```
java --version
```

A valid Java installation is required because Jenkins agents run using Java.

```
C:\Users\Amol Pathak>java --version
java 21.0.12.1 2026-08-18 LTS
Java(TM) SE Runtime Environment (build 21.0.12.1+1-LTS-4)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12.1+1-LTS-4, mixed mode, sharing)
```

Figure 1: Java installation verification on agent node

Step 2: Open Jenkins Dashboard

Log in to Jenkins using the controller node.

Navigate to:

Manage Jenkins → Nodes

This section is used to manage distributed build nodes.

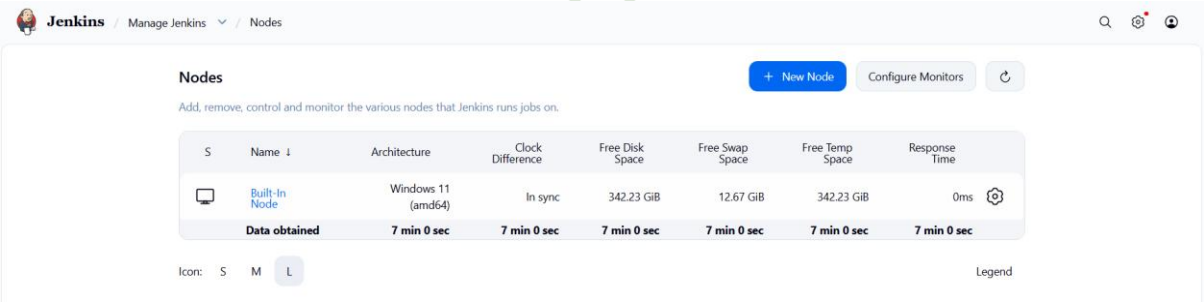


Figure 2: Jenkins Nodes management page

Step 3: Create a New Agent Node

Select:

New Node

Provide:

- Node Name
- Permanent Agent option

Then click **Create**.

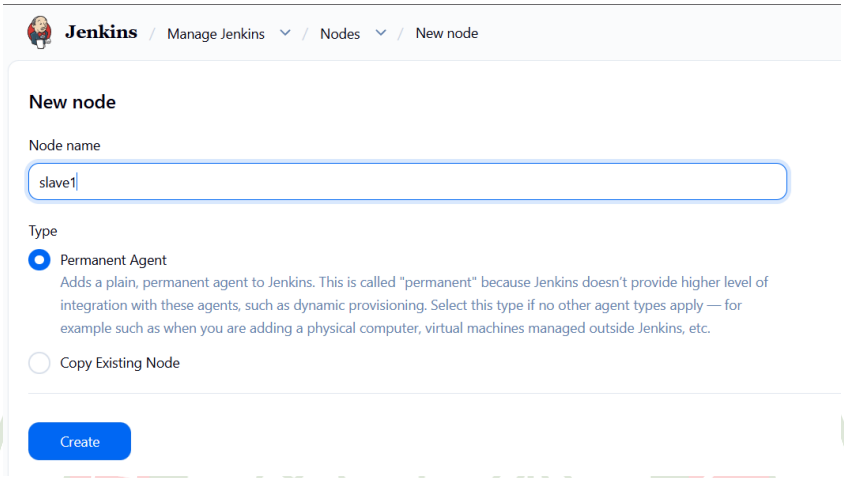


Figure 3: Creating a new Jenkins agent node

Step 4: Configure Agent Properties

Configure:

- Remote Root Directory
- Labels
- Number of Executors
- Usage Settings
- Launch Method

Example configuration:

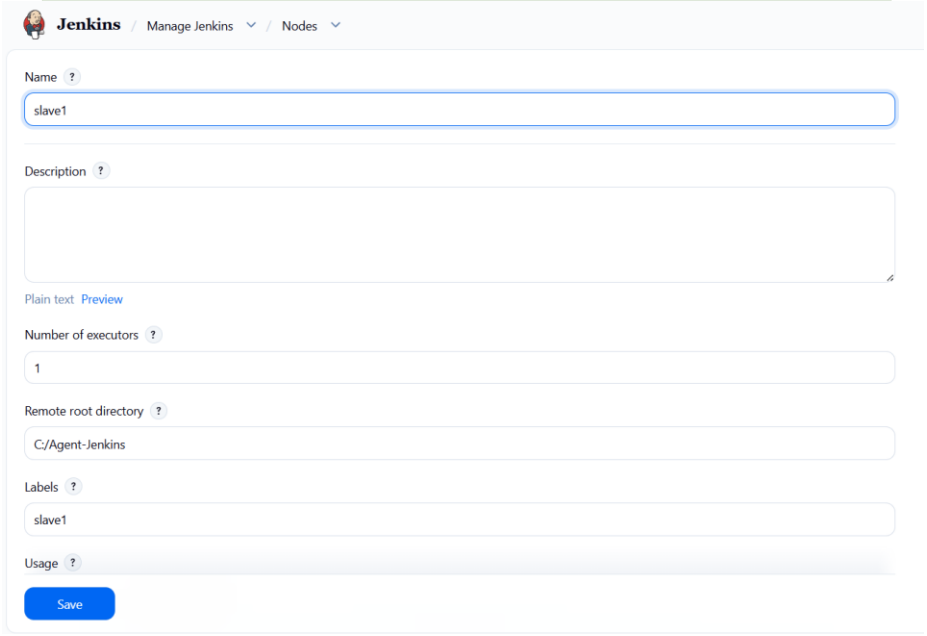


Figure 4: Agent node configuration screen

Step 5: Connect Agent to Controller

Select an appropriate launch method.

For inbound agent configuration:

1. Save the node configuration.
2. Open the agent page.
3. Download or launch the agent file.
4. Connect the agent to the Jenkins controller.

Once connected, the node status should change to:

Connected or Online

```
PS C:\Users\Amol Pathak> curl.exe -sO http://localhost:8081/jnlpJars/agent.jar
PS C:\Users\Amol Pathak> java -jar agent.jar -url http://localhost:8081/ -secret 3001aba73678b3660b79a146f6e0a4ab39816bd
dd0d023603f46c0d9d3ca0360 -name slave1 -webSocket -workDir "C:/Agent-Jenkins"
Sept 10, 2026 8:41:17 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using C:\Agent-Jenkins\remoting as a remoting work directory
Sept 10, 2026 8:41:17 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to C:\Agent-Jenkins\remoting
Sept 10, 2026 8:41:17 PM hudson.remoting.Launcher createEngine
INFO: Setting up agent: slave1
Sept 10, 2026 8:41:17 PM hudson.remoting.Engine startEngine
INFO: Using Remoting version: 3386.v353e57a_1b_ea_0
Sept 10, 2026 8:41:17 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using C:\Agent-Jenkins\remoting as a remoting work directory
Sept 10, 2026 8:41:17 PM hudson.remoting.Launcher$CuiListener status
INFO: WebSocket connection open
Sept 10, 2026 8:41:17 PM hudson.remoting.Launcher$CuiListener status
INFO: Connected
```

Figure 5: Successfully connected agent node

Step 6: Verify Node Status

Return to:

Manage Jenkins → Nodes

The newly created node should display:

- Online status
- Executor availability
- Node labels

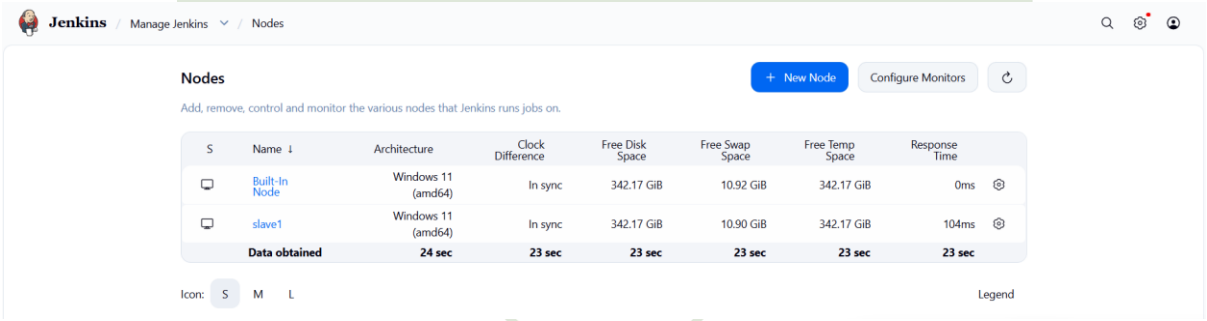


Figure 6: Agent node displayed as online

Step 7: Create a Build Job

Create a new Freestyle Project.

Under job configuration:

Restrict where this project can run

Specify the node label:

build-agent

This ensures the build executes on the configured agent.

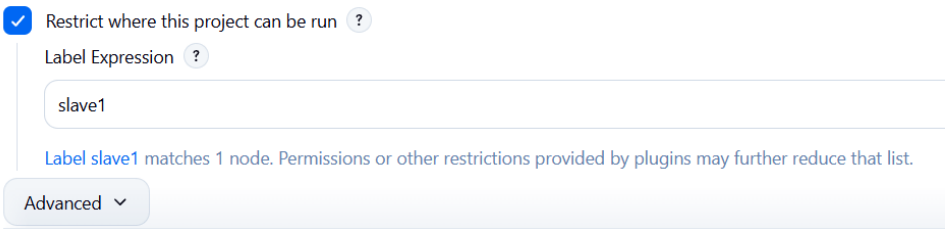


Figure 7: Build job restricted to agent node

Step 8: Execute the Build

Add a simple build step.

For Windows:

echo Build Executed on Jenkins Agent

For Linux:

echo "Build Executed on Jenkins Agent"

Save the configuration and click:

Build Now

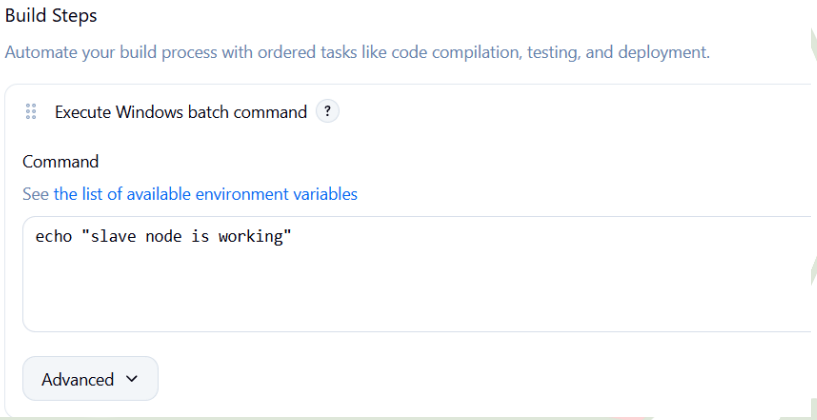


Figure 8: Build execution initiated

Step 9: Verify Console Output

Open the build result and select:

Console Output

The output should display:

Build Executed on Jenkins Agent

along with details indicating execution on the configured agent node.

A successful execution ends with:

Finished: SUCCESS



Figure 9: Successful build execution on agent node

Jenkins Build Workflow

The basic workflow demonstrated in this experiment can be summarized as:

Create Job → Configure Build Step → Save → Build Now → Console Output → Verify Success

Jenkins automates the execution of the configured build tasks, providing a basic demonstration of Continuous Integration.

Sample Build Command

echo “slave node is working”

Result:

The Jenkins Master-Slave architecture was successfully studied and implemented. An agent node was configured, connected to the Jenkins controller, and used to execute a build job. The build execution was successfully verified through the Jenkins console output.

Conclusion:

The experiment demonstrated how Jenkins can scale beyond a standalone installation by utilizing distributed agent nodes. The Master-Slave architecture improves scalability, workload distribution, resource utilization, and build performance by allowing multiple systems to participate in the build process. This architecture forms the foundation of large-scale Continuous Integration and Continuous Delivery environments.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY స్వయం తేజస్విన్ భవ 1979

Docker Architecture and Lifecycle

Aim:

To understand Docker architecture and the container lifecycle, install Docker, and execute Docker commands to manage images and interact with containers.

Objectives:

- To understand the fundamentals of containerization using Docker.
- To study the architecture and components of Docker.
- To understand the lifecycle of a Docker container.
- To install and configure Docker on a system.
- To execute commonly used Docker commands for image and container management.
- To interact with running containers and observe their behavior.

Hardware and Software Requirements

Hardware

- Computer or laptop.
- Minimum 4 GB RAM; 8 GB RAM or more is recommended.
- Stable internet connection.

Software

- Windows, Linux, or macOS operating system.
- Docker Desktop or Docker Engine.
- Command Prompt, PowerShell, Terminal, or Git Bash.
- Web browser.

Procedure

1. Download and install Docker Desktop or Docker Engine.
2. Verify that Docker has been installed successfully.
3. Study the major components of Docker architecture.
4. Understand the Docker container lifecycle.
5. Pull a Docker image from Docker Hub.
6. Create and start a container from the image.
7. View running and stopped containers.
8. Interact with a container using Docker commands.
9. Stop, restart, and remove containers.
10. Remove unused Docker images.
11. Record observations regarding image and container management.

Theory

1. Introduction to Docker

Docker is an open-source containerization platform that enables applications and their dependencies to be packaged into lightweight and portable containers.

Unlike traditional virtual machines, containers share the host operating system kernel, making them more efficient in terms of resource utilization and startup time.

Docker helps developers:

- Build applications consistently across environments.
- Simplify deployment processes.
- Improve scalability and portability.
- Isolate applications from one another.
- Reduce dependency-related issues.

2. Docker Architecture

Docker follows a client-server architecture consisting of several components.

Docker Client

The Docker Client is the interface through which users execute Docker commands.
Examples:

```
docker run
docker pull
docker ps
```

The client communicates with the Docker daemon to perform requested operations.

Docker Daemon

The Docker Daemon (dockerd) is responsible for:

- Building images.
- Managing containers.
- Handling Docker networks.
- Managing Docker volumes.
- Communicating with Docker clients.

Docker Host

The Docker Host is the machine on which Docker is installed and running.
It contains:

- Docker Engine
- Images
- Containers
- Networks
- Volumes

Docker Registry

A Docker Registry stores Docker images.
Examples include:

- Docker Hub
- Private Docker Registry
- Enterprise Container Registry

Docker images can be downloaded (pulled) or uploaded (pushed) to a registry.

3. Docker Architecture Diagram

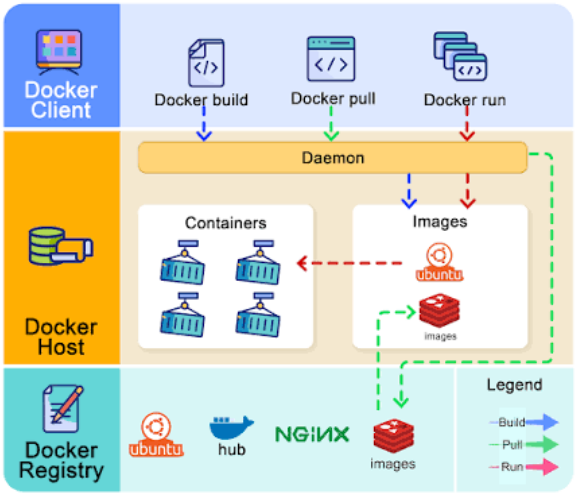


Figure 1: Docker Architecture

4. Docker Images

A Docker image is a read-only template used to create containers.

Characteristics:

- Contains application code and dependencies.
- Can be reused multiple times.
- Acts as a blueprint for containers.
- Stored locally or in registries.

Examples:

```
ubuntu
nginx
mysql
python
```

5. Docker Containers

A container is a running instance of a Docker image.

Containers provide:

- Process isolation.
- Application portability.
- Lightweight execution.
- Rapid startup.

Multiple containers can be created from the same image.

6. Docker Container Lifecycle

A Docker container passes through several states during its lifecycle.

Created: The container has been created but is not running.

Running: The container is actively executing processes.

Paused: The container processes are temporarily suspended.

Stopped: The container is no longer running but still exists on the system.

Removed: The container has been deleted from the Docker host.

Lifecycle representation:
Create → Run → Pause → Resume → Stop → Remove

7. Common Docker Commands

Command	Purpose
docker --version	Displays Docker version
docker pull <image>	Downloads an image
docker images	Lists available images
docker run <image>	Creates and starts a container
docker ps	Lists running containers
docker ps -a	Lists all containers
docker stop <container>	Stops a running container
docker start <container>	Starts a stopped container
docker exec -it <container> bash	Accesses a running container
docker rm <container>	Removes a container
docker rmi <image>	Removes an image

Execution Steps

Step 1: Install Docker
Download and install Docker Desktop from the official Docker website.
After installation, start Docker Desktop and ensure the Docker service is running.

Step 2: Verify Docker Installation
Open Command Prompt, PowerShell, or Terminal and execute:

```
docker --version
```

A successful installation displays the installed Docker version.

```
C:\Users\Amol Pathak>docker --version
Docker version 29.4.0, build 9d7ad9f
```

Figure 2: Docker version verification

Step 3: Verify Docker Service
Check whether Docker is functioning properly:

```
docker info
```

This command displays Docker Engine information and confirms communication with the Docker daemon.

Step 4: Download an Image
Pull the Ubuntu image from Docker Hub:

```
docker pull ubuntu
```

Docker downloads the required image layers and stores them locally.

```
C:\Users\Amol Pathak>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
06ad70e463aa: Pull complete
4e07a0f12b2c: Pull complete
8f70d2bfe91a: Download complete
Digest: sha256:f144425ff09be612d6d9ad965196e9cdc23dae1f42110a8a11a3e9a8198759f7
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

What's next:
    View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
```

Figure 3: Pulling Ubuntu image from Docker Hub

Step 5: View Available Images

Display all downloaded images:

```
docker images
```

The command shows:

- Repository name
- Tag
- Image ID
- Creation date
- Image size

```
C:\Users\Amol Pathak>docker images

IMAGE                     ID             DISK USAGE   CONTENT SIZE   EXTRA
postgis/postgis:15-3.3    a2fc46b52819       825MB          206MB   U
ubuntu:latest             f144425ff09b       162MB         45.6MB
```

Figure 4: Available Docker images

Step 6: Create and Run a Container

Create and start a container using the Ubuntu image:

```
docker run -it ubuntu bash
```

Explanation:

- -i : Interactive mode.
- -t : Allocates a terminal.
- ubuntu : Image name.
- bash : Starts the Bash shell.

The terminal prompt changes to indicate execution inside the container.

Step 7: Interact with the Container

Execute commands within the running container:

```
pwd
ls
whoami
```

These commands demonstrate interaction with the containerized environment.

```
C:\Users\Amol Pathak>docker run -it ubuntu bash
root@65c1d535cb71:/# pwd
/
root@65c1d535cb71:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@65c1d535cb71:/# whoami
root
```

Figure 5: Executing commands inside the container

Step 8: Exit the Container

Exit the running container:

```
exit
```

The container stops after exiting the shell session.

Step 9: View Container Lifecycle Status

Display all containers:

```
docker ps -a
```

Observe the container status:

- Up (Running)
- Exited (Stopped)

This demonstrates lifecycle state transitions.

```
C:\Users\Amol Pathak>docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS                     PORTS
          NAMES
65c1d535cb71   ubuntu                 "bash"                   3 minutes ago   Exited (0) 9 seconds ago
          bold_solomon
b811f072957a   postgis/postgis:15-3.3  "docker-entrypoint.s…"  5 months ago    Exited (255) 8 days ago    0.0.0.0:5432
->5432/tcp   agrishield-db
```

Figure 6: Container status after execution

Step 10: Restart the Container

Start the stopped container again:

```
docker start <container_id>
```

Verify the status:

```
docker ps
```

```
C:\Users\Amol Pathak>docker start 65c1d535cb71
65c1d535cb71

C:\Users\Amol Pathak>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED         STATUS          PORTS     NAMES
65c1d535cb71   ubuntu    "bash"    4 minutes ago   Up 10 seconds             bold_solomon
```

Figure 7: Restarted container

Step 11: Access the Running Container

Connect to the container:

```
docker exec -it <container_id> bash
```

This opens an interactive shell session within the container.

```
C:\Users\Amol Pathak>docker exec -it 65c1d535cb71 bash
root@65c1d535cb71:/# whoami
root
```

Figure 8: Accessing a running container

Step 12: Remove the Container

Stop the container if necessary:

```
docker stop <container_id>
```

Remove it:

```
docker rm <container_id>
```

This deletes the container from the Docker host.

```
C:\Users\Amol Pathak>docker stop 65c1d535cb71
65c1d535cb71

C:\Users\Amol Pathak>docker rm 65c1d535cb71
65c1d535cb71
```

Figure 9: Container removal

Step 13: Remove the Image

Delete the image:

docker rmi ubuntu

This removes the image from local storage.

```
C:\Users\Amol Pathak>docker rmi ubuntu
Untagged: ubuntu:latest
Deleted: sha256:f144425ff09be612d6d9ad965196e9cdc23dae1f42110a8a11a3e9a8198759f7
```

Figure 10: Image removal

Docker Container Lifecycle Demonstration

The lifecycle of a Docker container is observed during this experiment.
This demonstrates how Docker manages containers throughout their existence.

Sample Commands

```
docker pull ubuntu
docker images
docker run -it ubuntu bash
docker ps -a
docker stop <container_id>
docker rm <container_id>
docker rmi ubuntu
```

Result

Docker was successfully installed and configured. Docker images were downloaded, containers were created and executed, and various Docker commands were used to manage images and interact with containers. The Docker container lifecycle was observed and verified successfully.

Conclusion

The experiment provided practical understanding of Docker architecture and containerization concepts. Docker images and containers were managed using standard Docker commands, and the complete container lifecycle—from creation to removal—was studied. The experiment demonstrated how Docker enables lightweight, portable, and efficient application deployment through container technology.

Dockerfile and Image Building

Aim

To study Dockerfile instructions and use a Dockerfile to create a container image for a simple web application.

Objectives

- To understand the purpose and structure of a Dockerfile.
- To study commonly used Dockerfile instructions.
- To create a simple web application that can be served using Nginx.
- To write a Dockerfile for the web application.
- To build a Docker image using the Dockerfile.
- To run a container from the newly created image.
- To access and verify the web application through a web browser.

Hardware and Software Requirements

Hardware

- Computer or laptop.
- Minimum 4 GB RAM; 8 GB RAM or more is recommended.
- Stable internet connection.

Software

- Windows, Linux, or macOS operating system.
- Docker Desktop or Docker Engine.
- PowerShell, Command Prompt, or Terminal.
- Text editor such as Notepad or Visual Studio Code.
- Web browser.

Procedure

1. Verify that Docker is installed and running.
2. Create a directory for the sample web application.
3. Create an HTML file containing the web-page content.
4. Create a file named Dockerfile in the application directory.
5. Add appropriate Dockerfile instructions.
6. Build a Docker image from the Dockerfile.
7. Verify that the newly created image is available locally.
8. Create and run a container using the image.
9. Map the container's web-server port to a port on the host system.
10. Open the application in a web browser and verify its output.

Theory

1. What is a Dockerfile?

A Dockerfile is a text file containing a sequence of instructions used by Docker to construct an image.

Instead of manually configuring a container every time an application needs to be deployed, the required environment and application files can be described in a Dockerfile.

A Dockerfile therefore acts as a reproducible set of instructions for creating a Docker image.

2. Dockerfile Instructions

Some commonly used Dockerfile instructions are:

Instruction	Purpose
FROM	Specifies the base image for the new image
WORKDIR	Sets the working directory inside the image
COPY	Copies files from the build context into the image
ADD	Adds files or directories to the image
RUN	Executes a command while building the image
CMD	Specifies the default command executed when a container starts
ENTRYPOINT	Defines the primary executable for the container
EXPOSE	Documents the port intended to be used by the application
ENV	Defines environment variables
LABEL	Adds metadata to the image

For this experiment, the basic instructions FROM and COPY are sufficient to create the sample web application image.

3. Base Image

The FROM instruction specifies the starting point for the image.

For this experiment:

```
FROM nginx
```

The official Nginx image provides a ready-to-use web server environment.

4. Copying Application Files

The COPY instruction transfers files from the Docker build context into the image.

For the sample application:

COPY index.html /usr/share/nginx/html

The HTML file is copied into the default directory from which Nginx serves static web content.

5. Docker Image Building Process

The Docker image is created using:

```
docker build -t myapp .
```

Here:

- docker build starts the image-building process.
- -t myapp assigns the name myapp to the image.
- . specifies the current directory as the build context.

During the build, Docker reads the Dockerfile and executes its instructions to create the image.

6. Running the Image as a Container

After building the image, a container can be created using:

```
docker run -p 8080:80 myapp
```

The port mapping means:

Host Port 8080 → Container Port 80

Therefore, requests sent to port 8080 on the host are forwarded to Nginx running on port 80 inside the container.

Execution Steps

Step 1: Create the Application Directory

Open PowerShell or Terminal and create a directory for the web application:

```
mkdir myapp
cd myapp
```

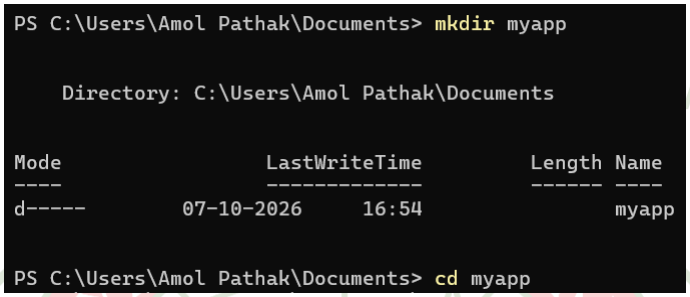


Figure 1: Creating and entering the web application directory

Step 2: Create the HTML File

Create a file named index.html.

For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>My Docker App</title>
</head>
<body>
    <h1>Hello from Docker!</h1>
    <p>This web page is running inside a Docker container.</p>
</body>
</html>
```

Verify that the file exists:

```
ls
```

Step 3: Create the Dockerfile

Inside the same directory, create a file named:

```
Dockerfile
```

The Dockerfile should contain:

```
FROM nginx
COPY index.html /usr/share/nginx/html
```

The first instruction selects Nginx as the base image, while the second copies the application's HTML file into the Nginx web directory.

The reference experiment uses this same minimal Dockerfile structure for the sample application.

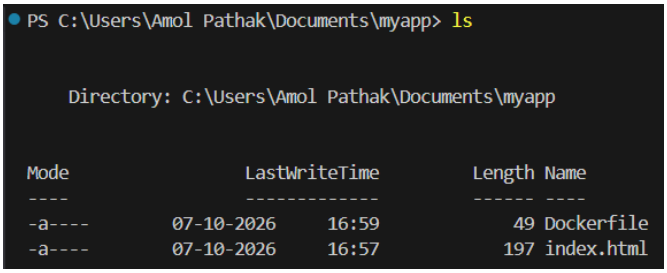


Figure 2: Sample web application file and Dockerfile

Step 4: Verify Docker

Before building the image, verify that Docker is available:

```
docker --version
```

Optionally, obtain detailed Docker information using:

```
docker info
```

Step 5: Build the Docker Image

From the directory containing the Dockerfile, execute:

```
docker build -t myapp .
```

Docker reads the Dockerfile and generates the myapp image.

The build process should indicate successful completion.

The reference execution shows the image being successfully built and tagged as myapp:latest.



Figure 3: Successfully building the Docker image

Step 6: Verify the Image

List the locally available Docker images:

```
docker images
```

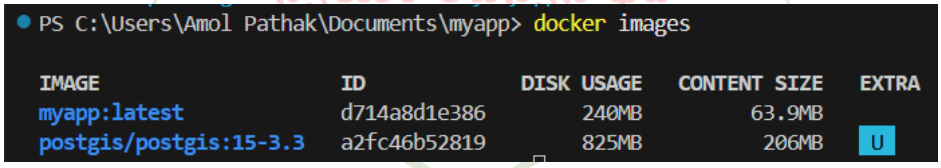


Figure 4: Docker image myapp listed among local images

Step 7: Run the Container

Start a container from the newly created image:

```
docker run -p 8080:80 myapp
```

Here, port 8080 of the host is mapped to port 80 of the Nginx container.

The reference experiment uses the same port mapping to run the myapp image.

```
PS C:\Users\Amol Pathak\Documents\myapp> docker run -p 8080:80 myapp
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/07 11:39:12 [notice] 1#1: using the "epoll" event method
2026/10/07 11:39:12 [notice] 1#1: nginx/1.31.6
2026/10/07 11:39:12 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/10/07 11:39:12 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/10/07 11:39:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/10/07 11:39:12 [notice] 1#1: start worker processes
2026/10/07 11:39:12 [notice] 1#1: start worker process 29
2026/10/07 11:39:12 [notice] 1#1: start worker process 30
2026/10/07 11:39:12 [notice] 1#1: start worker process 31
2026/10/07 11:39:12 [notice] 1#1: start worker process 32
2026/10/07 11:39:12 [notice] 1#1: start worker process 33
2026/10/07 11:39:12 [notice] 1#1: start worker process 34
```

Figure 5: Running the Docker container

Step 8: Access the Web Application

Open a web browser and enter:

`http://localhost:8080`

The HTML page created in Step 2 should be displayed.

This confirms that:

1. The Dockerfile was processed successfully.
2. The Docker image was created.
3. A container was started from the image.
4. Nginx is serving the HTML file.
5. The application is accessible through the mapped host port.

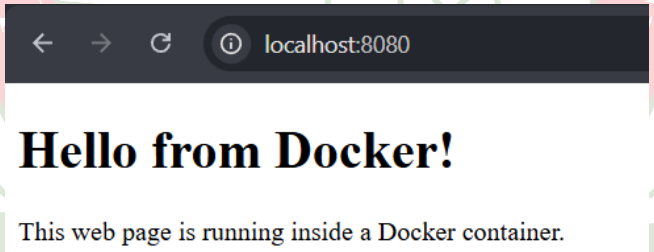


Figure 6: Sample web application running in the Docker container

Step 9: Verify the Running Container

Open another terminal and execute:

`docker ps`

This displays the currently running containers.

The output should show the myapp container and its port mapping.

```
PS C:\Users\Amol Pathak\Documents\myapp> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                     NAMES
503f98f95df4   myapp     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   agitated_proskuriak
ova
```

Figure 7: Running container and port mapping

Dockerfile Used in the Experiment

```
FROM nginx
COPY index.html /usr/share/nginx/html
```

Commands Used

```
docker --version
docker info
docker build -t myapp .
docker images
docker run -p 8080:80 myapp
docker ps
```

Dockerfile Workflow

The complete workflow can be summarized as:

1. Create Web Page
2. Create Dockerfile
3. Specify Base Image
4. Copy Application Files
5. Build Docker Image
6. Run Container
7. Map Ports
8. Access Web Application

Result

A Dockerfile was created using appropriate Dockerfile instructions, and a Docker image named myapp was successfully built. A container was launched from the image, and the sample web application was accessed through the mapped host port.

Conclusion

The experiment provided practical understanding of Dockerfile-based image creation. The FROM and COPY instructions were used to construct an Nginx-based image containing a sample HTML application. The resulting image was built and executed as a container, demonstrating how Dockerfiles provide a repeatable method for packaging and deploying web applications.

Kubernetes Application Deployment

Aim

To understand Kubernetes cluster architecture and deploy a containerized application on a Kubernetes cluster using Deployments and Services.

Objectives

- To understand the fundamental concepts and architecture of Kubernetes.
- To study the roles of the Control Plane and Worker Nodes.
- To understand how Kubernetes manages containerized applications.
- To deploy a containerized application using a Kubernetes Deployment.
- To expose the deployed application using a Kubernetes Service.
- To execute Kubernetes commands for managing and monitoring application resources.
- To verify the successful deployment and accessibility of the application.

Hardware and Software Requirements

Hardware

- Computer or laptop with at least 4 GB RAM; 8 GB or more is recommended.
- Stable internet connection.

Software

- Windows, Linux, or macOS operating system.
- Docker Desktop with Kubernetes enabled, or Minikube.
- kubectl command-line tool.
- Web browser.
- Terminal, PowerShell, or Command Prompt.

Procedure

1. Install Docker Desktop or Minikube and configure a Kubernetes cluster.
2. Install and verify the kubectl command-line tool.
3. Check the status of the Kubernetes cluster and its nodes.
4. Create a Deployment configuration for a containerized application.
5. Apply the configuration to the Kubernetes cluster.
6. Verify the Deployment and the creation of application Pods.
7. Expose the application using a Kubernetes Service.
8. Access the application through the configured Service.
9. Execute Kubernetes commands to inspect and manage the deployed resources.
10. Record the observations and verify successful application deployment.

Theory

1. Introduction to Kubernetes

Kubernetes is an open-source container orchestration platform used to deploy, manage, and scale containerized applications.

It automates several operational tasks, including container scheduling, application scaling, service discovery, and recovery from certain container or node failures.

Kubernetes helps organizations manage applications consistently across development, testing, and production environments.

2. Kubernetes Architecture

A Kubernetes cluster consists of a Control Plane and one or more Worker Nodes.

Control Plane

The Control Plane manages the overall state of the cluster and makes decisions about application deployment and resource allocation.

Its major components include:

- **API Server:** Provides the interface through which users and cluster components communicate.
- **Scheduler:** Selects suitable nodes for newly created Pods.
- **Controller Manager:** Runs controllers that help maintain the desired cluster state.
- **etcd:** Stores the cluster's configuration and state data.

Worker Nodes

Worker Nodes run the application workloads assigned by Kubernetes.

Their major components include:

- **Kubelet:** Ensures that assigned Pods are running as specified.
- **Container Runtime:** Runs containers, for example, containerd.
- **Kube-proxy or equivalent networking implementation:** Supports Service networking, depending on the cluster configuration.

3. Important Kubernetes Objects

Pod: The smallest deployable unit in Kubernetes. A Pod contains one or more closely associated containers.

Deployment: Manages replicated application Pods and supports updates and replacement of Pods when necessary.

Service: Provides a stable network endpoint for accessing a group of Pods.

Namespace: Organizes resources within a cluster.

ConfigMap: Stores non-sensitive configuration data separately from application images.

4. Kubernetes Deployment

A Deployment describes the desired state of an application, including the container image and the number of replicas.

Kubernetes works to maintain the specified state. For example, if a Deployment requests two replicas, Kubernetes attempts to keep two corresponding Pods available.

5. Kubernetes Service

A Service provides a consistent way to access application Pods, even when individual Pods are replaced.

Common Service types include:

- **ClusterIP:** Makes a Service accessible within the cluster.
- **NodePort:** Exposes a Service through a port on cluster nodes.
- **LoadBalancer:** Integrates with a supported load-balancing environment.

For a local demonstration, a NodePort Service or kubectl port-forward can be used to access the application.

6. Benefits of Kubernetes

- Automated container scheduling.
- Application scaling and load distribution.
- Desired-state management.
- Service discovery and networking.
- Replacement of failed Pods when possible.
- Controlled application updates and rollbacks.

Execution Steps

Step 1: Start the Kubernetes Cluster

Start Docker Desktop with Kubernetes enabled, or start a local Minikube cluster.
For Minikube, execute:

```
minikube start
```

Verify that kubectl is available:

```
kubectl version --client
```

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

Figure1: Kubernetes cluster startup and kubectl version verification.

Step 2: Verify Cluster Status

Check the available cluster nodes:

```
kubectl get nodes
```

The node should show a Ready status when the cluster is functioning correctly.
Check the cluster information:

```
kubectl cluster-info
```

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   47s   v1.37.0
PS C:\Users\Amol Pathak\Documents\myapp> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:65177
CoreDNS is running at https://127.0.0.1:65177/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

Figure 2: Kubernetes node displayed as Ready.

Step 3: Create a Deployment

Deploy a sample Nginx web server:

```
kubectl create deployment web-app --image=nginx
```

This command creates a Deployment named web-app using the Nginx container image.
Kubernetes retrieves the image if necessary and schedules a Pod to run the application.

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl create deployment web-app --image=nginx
deployment.apps/web-app created
```

Figure 3: Creating the web-app Deployment.

Step 4: Verify the Deployment and Pods

Inspect the Deployment:

```
kubectl get deployments
```

List the Pods:

```
kubectl get pods
```

For more detailed information about the Deployment, use:

```
kubectl describe deployment web-app
```

Wait until the Pod reaches the Running state and its containers are ready.

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
web-app   0/1     1            0           22s
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get pods
NAME                       READY   STATUS    RESTARTS   AGE
web-app-858648cd44-vcjwg   1/1     Running   0          29s
```

Figure 4: Deployment and Pod status showing successful creation.

Step 5: Expose the Application

Expose the Deployment through a Service:

```
kubectl expose deployment web-app --type=NodePort --port=80
```

This creates a Service named web-app that directs traffic to the Nginx Pods on port 80.

Verify the Service:

```
kubectl get services
```

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl expose deployment web-app --type=NodePort --port=80
service/web-app exposed
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get services
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP        2m10s
web-app      NodePort    10.110.136.51   <none>        80:30582/TCP   8s
```

Figure 5: Kubernetes Service configuration and assigned NodePort.

Step 6: Access the Application

For a convenient local test, use port forwarding:

kubectl port-forward service/web-app 8080:80

Keep this command running in the terminal. Open a web browser and visit:

http://localhost:8080

The default Nginx welcome page should appear, confirming that the application is accessible.

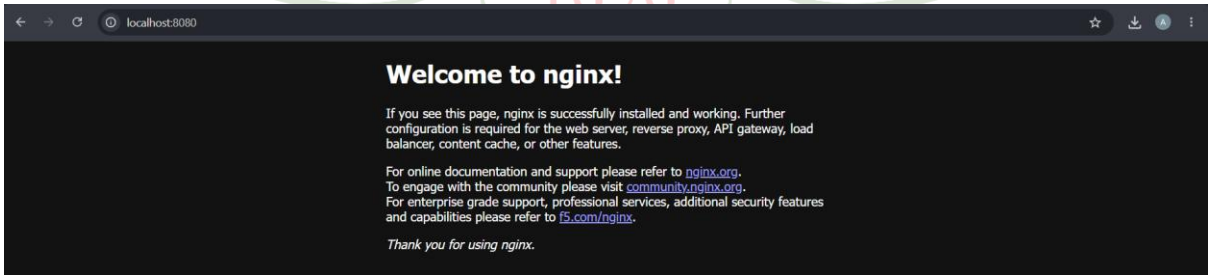


Figure 6: Nginx welcome page displayed in the browser.

Step 7: Scale the Application

Increase the number of application replicas to two:

```
kubectl scale deployment web-app --replicas=2
```

Verify the result:

```
kubectl get pods
kubectl get deployments
```

Kubernetes should create additional Pods to match the desired replica count.

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl scale deployment web-app --replicas=2
deployment.apps/web-app scaled
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get pods
NAME                       READY   STATUS    RESTARTS   AGE
web-app-858648cd44-r2vb5   1/1     Running   0          8s
web-app-858648cd44-vcjwg   1/1     Running   0          17m
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
web-app   2/2     2            2           17m
```

Figure 7: Two application Pods running under the Deployment.

Step 8: Inspect Application Resources

Display the Service details:

```
kubectl describe service web-app
```

View recent cluster events:

```
kubectl get events
```

These commands help examine resource configuration and identify possible deployment issues.

Step 9: Clean Up Resources

After completing the experiment, delete the Deployment and Service:

```
kubectl delete service web-app
kubectl delete deployment web-app
```

Verify that the resources have been removed:

```
kubectl get deployments
kubectl get services
```

```
PS C:\Users\Amol Pathak\Documents\myapp> kubectl delete service web-app
service "web-app" deleted from default namespace
PS C:\Users\Amol Pathak\Documents\myapp> kubectl delete deployment web-app
deployment.apps "web-app" deleted from default namespace
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get deployments
No resources found in default namespace.
PS C:\Users\Amol Pathak\Documents\myapp> kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   21m
```

Figure 8: Successful deletion of the Deployment and Service.

Important Kubernetes Commands

Command	Purpose
kubectl get nodes	Lists cluster nodes
kubectl cluster-info	Displays cluster endpoint information
kubectl create deployment	Creates an application Deployment
kubectl get deployments	Lists Deployments
kubectl get pods	Lists Pods
kubectl describe deployment	Displays detailed Deployment information
kubectl expose deployment	Creates a Service for a Deployment
kubectl get services	Lists Services
kubectl scale deployment	Changes the desired replica count
kubectl delete deployment	Deletes a Deployment

Result

A containerized Nginx application was deployed on a Kubernetes cluster. The Deployment, Pods, and Service were inspected, the application was accessed through a local port mapping, and the application was scaled to multiple replicas.

Conclusion

The experiment demonstrated the basic workflow of deploying and managing a containerized application using Kubernetes. The roles of Deployments, Pods, and Services were studied, and practical commands were used to inspect cluster resources, expose the application, scale replicas, and clean up resources.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

Puppet Configuration Management

Aim

To install and configure Puppet, a pull-based Software Configuration Management (SCM) and provisioning tool, and understand how it automates system configuration across managed nodes.

Objectives

- Understand the principles of Software Configuration Management.
- Learn the architecture and working mechanism of Puppet.
- Install and configure Puppet components.
- Define configuration requirements using Puppet manifests.
- Apply configurations on managed nodes and verify the results.

Hardware and Software Requirements

- **Hardware:** Computer with at least 4 GB RAM and a stable network connection.
- **Software:** Linux-based operating system or virtual machines, Puppet packages, terminal, and administrative privileges.

Theory

Software Configuration Management

Software Configuration Management (SCM) is the process of maintaining consistent system configurations across multiple machines. It helps automate software installation, service management, file creation, and configuration updates.

Puppet

Puppet is a configuration management and provisioning tool that allows administrators to define the desired state of a system using a declarative language. Instead of manually configuring every machine, administrators specify the required state in Puppet manifests, and Puppet works to maintain that state.

Pull-Based Configuration Management

Puppet commonly uses a pull-based approach in which managed nodes periodically contact the Puppet server to retrieve their configuration catalog and apply the required changes. This allows each node to synchronize its configuration with the centrally defined policies.

Puppet Architecture

The main components of Puppet are:

1. **Puppet Server:** Stores configuration information and compiles catalogs for managed nodes.
2. **Puppet Agent:** Runs on managed nodes, requests configuration catalogs, and applies them locally.
3. **Manifest:** A file containing configuration instructions written in Puppet's declarative language.
4. **Catalog:** A compiled description of the resources and desired states that an agent must enforce.

5. **Resources:** Individual configuration items, such as files, packages, users, and services.

Procedure

Step 1: Prepare the Environment

Set up a Linux system to act as the Puppet server and another system or virtual machine to act as the Puppet agent. For a basic demonstration, a single Linux machine can also be used to explore Puppet manifests locally.

Ensure that the machines can communicate over the network and that the required administrative permissions are available.

Step 2: Install Puppet

Install Puppet using the package repository and installation instructions appropriate for the selected Linux distribution and Puppet version.

After installation, verify the available Puppet commands:

```
puppet --version
```

If using a server-agent setup, install and configure the Puppet Server on the server machine and the Puppet Agent on the managed machine.

```
amol_pathak@AmolPathakLOQ:~$ puppet --version
8.4.0
```

Figure 1: Puppet installation verification

Step 3: Configure the Puppet Server and Agent

In a server-agent environment:

1. Configure the Puppet Server with its hostname and network settings.
2. Configure the agent to identify the correct Puppet Server.
3. Ensure the server and agent can communicate.
4. Start and enable the required services.
5. Complete certificate signing and verification so the agent can securely communicate with the server.

The exact configuration commands depend on the Linux distribution and Puppet version being used.

Step 4: Create a Puppet Manifest

Create a manifest file named site.pp to specify a desired configuration. The following example ensures that a file named hello.txt exists and contains a message.

```
file { '/tmp/hello.txt':
  ensure  => file,
  content => "Hello from Puppet!\n",
}
```

Save the manifest in the appropriate directory for the selected setup. In a standard Puppet Server configuration, site manifests are commonly maintained under the environment's manifests directory.

```
  GNU nano 7.2
file { '/tmp/puppet-demo.txt':
  ensure  => file,
  content => "Hello from Puppet!\n",
}
```

Figure 2: Manifest file content

Step 5: Apply the Configuration

In a configured server-agent environment, run the Puppet Agent on the managed node to request and apply its catalog:

```
sudo puppet agent --test
```

The agent contacts the server, retrieves its catalog, and applies the defined configuration. The server-agent setup must be configured correctly before this step can succeed.

For a standalone demonstration without a Puppet Server, a local manifest can be applied directly:

```
sudo puppet apply site.pp
```

This standalone command is an alternative for testing Puppet manifests; it does not demonstrate the full pull-based server-agent workflow.

```
amol_pathak@AmolPathakLOQ:~$ sudo puppet apply manifest_demo.pp
Notice: Compiled catalog for amolpathakloq.localdomain in environment production in 0.01 seconds
Notice: /Stage[main]/Main/File[/tmp/puppet-demo.txt]/ensure: defined content as '{sha256}ffdaae7a2b41b49949d4ed4b04cf76fdaf54e76a87
22109e47316f497f7ddce2'
Notice: Applied catalog in 0.02 seconds
```

Figure 2: Output of the command

Step 6: Verify the Configuration

Check whether the file was created with the expected content:

cat /tmp/hello.txt

Expected output:

Hello from Puppet!

Run the configuration again to observe Puppet's desired-state behaviour. If the file already matches the manifest, Puppet should generally report that no change is required.

Step 7: Review the Result

Inspect the command output for resource changes, errors, or successful application. In a server-agent setup, also verify the agent's connection to the server and ensure that the required certificate has been signed.

Result

Puppet was installed and configured for configuration management. A Puppet manifest was created and applied to ensure that the required file and its contents matched the declared state.

Conclusion

This experiment demonstrated Puppet's role in automating system provisioning and configuration management. It also illustrated the pull-based server-agent workflow, the use of manifests to define desired system states, and the verification of configuration changes.
resources.

Puppet Blocks and Modules

Aim

To study Software Configuration Management and provisioning using Puppet, with emphasis on manifests, modules, classes, and functions.

Objectives

- Understand the role of Puppet in configuration management and provisioning.
- Learn the structure and purpose of Puppet manifests.
- Organize configuration code using modules and classes.
- Understand how Puppet functions help process data and perform configuration-related operations.
- Implement and verify a sample configuration using Puppet.

Hardware and Software Requirements

- **Hardware:** Computer with at least 4 GB RAM.
- **Software:** Linux operating system, Puppet installation, terminal, and a text editor.
- **Privileges:** Administrative access for applying system-level configurations.

Theory

1. Puppet Manifest

A manifest is a file containing Puppet code that describes the desired state of system resources. Manifests generally use the .pp extension. They can declare resources such as files, packages, users, and services.

Example:

```
file { '/tmp/puppet-demo.txt':
  ensure  => file,
  content => "Puppet configuration example\n",
}
```

This resource ensures that the specified file exists and contains the given text.

2. Puppet Module

A module is a structured collection of Puppet code used to organize and reuse configuration logic. It can contain manifests, files, templates, and other supporting resources.

A typical module directory contains:

```
hello_module/manifests/init.pp
```

Modules make configurations easier to maintain and reuse across multiple systems.

3. Puppet Class

A class groups related resources into a reusable configuration unit. Classes can be declared when their configuration is required, making it easier to manage multiple resources together.

Example:

```
class demo_config {
  file { '/tmp/puppet-class.txt':
    ensure  => file,
    content => "Created using a Puppet class\n",
```

```
  }
}
```

The class declares the desired state of a file. It must be declared or included before Puppet applies its resources.

4. Puppet Function

A Puppet function performs an operation on supplied values and returns a result. Functions can be used to transform data, inspect values, and support decisions within Puppet code.

For example, the built-in upcase function converts a string to uppercase:

```
$message = upcase('hello puppet')
notice($message)
```

Expected output includes:

```
HELLO PUPPET
```

The notice function displays a message during catalog compilation or manifest evaluation, depending on the execution context.

Procedure

Step 1: Verify Puppet Installation

Open the terminal and check whether Puppet is installed.

```
puppet --version
```

If Puppet is not installed, install it using the appropriate instructions for the Linux distribution and Puppet version.

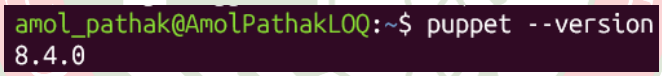


Figure 1: Puppet installation verification

Step 2: Create and Apply a Manifest

Create a file named manifest_demo.pp:

```
file { '/tmp/puppet-demo.txt':
  ensure  => file,
  content => "Puppet manifest executed successfully\n",
}
```

Apply the manifest:

```
sudo puppet apply manifest_demo.pp
```

Verify the file contents:

```
cat /tmp/puppet-demo.txt
```

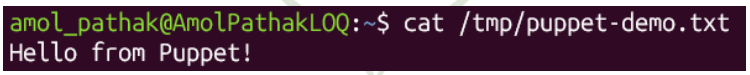


Figure 2: Verifying file content

Step 3: Create a Puppet Module

Create the module directory and its manifest:

```
mkdir -p hello_module/manifests
```

Create hello_module/manifests/init.pp with the following content:

```
class hello_module {
  file { '/tmp/puppet-module.txt':
    ensure  => file,
```

```
    content => "Configuration managed by a module\n",
  }
}
```

This defines a class named hello_module inside the module's main manifest.

Step 4: Apply the Module Class

Create a separate file named module_demo.pp:

```
include hello_module
```

Apply the class by specifying the module directory as a module path:

```
sudo puppet apply --modulepath=. module_demo.pp
```

Run this command from the directory containing hello_module.

Verify the result:

```
cat /tmp/puppet-module.txt
```

```
amol_pathak@AmolPathakLOQ:~$ sudo puppet apply --modulepath=. module_demo.pp
Notice: Compiled catalog for amolpathakloq.localdomain in environment production in 0.02 seconds
Notice: /Stage[main]/Hello_module/File[/tmp/puppet-module.txt]/ensure: defined content as '{sha256}591f2f6d363322c5912ef0d476d922b0
8487f73c1a0ff32e11993e6f92ef57d0'
Notice: Applied catalog in 0.02 seconds
```

```
amol_pathak@AmolPathakLOQ:~$ cat /tmp/puppet-module.txt
Configuration managed by a module
```

Figure 3: Module class output

Step 5: Demonstrate a Puppet Function

Create a file named function_demo.pp:

```
$message = upcase('hello puppet')
notice($message)
```

Execute the manifest:

```
puppet apply function_demo.pp
```

Observe the output and verify that the string has been converted to uppercase.

```
amol_pathak@AmolPathakLOQ:~$ puppet apply function_demo.pp
Notice: Scope(Class[main]): HELLO PUPPET
Notice: Compiled catalog for amolpathakloq.localdomain in environment production in 0.02 seconds
Notice: Applied catalog in 0.01 seconds
```

Figure 4: Puppet function output

Result

Puppet manifests, modules, classes, and built-in functions were demonstrated. The sample configurations were applied and verified successfully.

Conclusion

This experiment illustrated how Puppet supports Software Configuration Management and provisioning through declarative manifests, reusable modules, classes, and functions. These building blocks help organize configuration logic, reduce manual effort, and maintain consistent system states.